Dal punto nel piano al classificatore intelligente

Una dispensa su strutture dati, distanze e l'algoritmo K-Nearest Neighbors

---

Prerequisiti: saper leggere un programma C++ con un `main`
Obiettivo: capire e saper scrivere un classificatore KNN da zero
Livello: introduttivo — nessuna formula spaventosa, promesso.

S. Capolupo
Dispensa a uso didattico libero

Indice

1 Il problema: riconoscere le cose **2**

2 Strutture dati in C++ **2**
2.1 Variabili sparse 2
2.2 La `struct`: un contenitore con un nome 2
2.3 Array in C++: sequenze di elementi dello stesso tipo 3
2.4 Costanti simboliche: `const` e `#define` 4
2.5 Array di `struct` 4

3 Passare dati a una funzione: puntatori e array **4**
3.1 Le funzioni come scatole nere 4
3.2 Passaggio di array a una funzione 5
3.3 Passaggio di `struct` a una funzione 5

4 La distanza euclidea **5**
4.1 Cos'è una distanza? 5
4.2 Generalizzazione a n dimensioni 6
4.3 Implementazione in C++ 6

5 Ordinare dati: `sort` e `pair` **7**
5.1 Il problema dell'ordinamento 7
5.2 Tenere insieme distanza ed etichetta: `pair` 7

6 Contare i voti: `map` **8**
6.1 Il problema del conteggio 8
6.2 La `map`: un dizionario chiave-valore 8
6.3 Trovare il massimo in una `map` 8

7 L'algoritmo KNN **9**
7.1 Schema generale 9
7.2 La funzione `knn_predict` 9
7.3 Il programma completo 10

8 Leggere l'output e capire cosa succede **11**

9 Scegliere K: un parametro con carattere **12**

10 Limiti e sviluppi possibili **12**

1. Il problema: riconoscere le cose

Immaginate una cassetta di frutta mista tra mele e kiwi. Prendete un frutto a caso: lo osservate e vorreste sapere: *è una mela o un kiwi?*

Questo è il problema che risolve un algoritmo **classificatore**: dato un oggetto descritto da alcune misure o caratteristiche numeriche, assegnargli un'etichetta tra quelle note.

Il classificatore KNN — acronimo di *K Nearest Neighbors*, ovvero "K vicini più prossimi" — risolve il problema con un'idea tanto semplice da sembrare quasi ovvia:

"Guarda i K oggetti noti più simili al tuo
e adotta l'etichetta che compare più spesso."

Prima di arrivare al codice che implementa questa idea, dobbiamo comprendere i mattoni necessari: capire come rappresentare dati in C++, cosa significa "distanza" in senso matematico, e come organizzare la logica di classificazione. Procediamo per gradi.

2. Strutture dati in C++

2.1 Variabili sparse

Per non perdere completamente l'analogia con la frutta, poniamo che ogni frutto presente nella cassetta sia un punto nel piano cartesiano. Supponiamo pertanto di voler rappresentare un punto nel piano, come un'entità descritta da due coordinate x e y.

Il primo istinto potrebbe essere:

```cpp
int main() {
    double x = 3.0;
    double y = 5.0;
    // ... ma e' davvero un "punto"?
}
```

Funziona, certamente, ma è scomodo: se dovessimo rappresentare 500 punti, avremmo bisogno di 1000 variabili `x1`, `y1`, `x2`, `y2`...Il codice diventerebbe illeggibile e ingestibile.

2.2 La `struct`: un contenitore con un nome

In C++ la parola chiave `struct` permette di creare un nuovo tipo di dato che raggruppa variabili correlate tra loro sotto un singolo nome:

```cpp
struct Punto {
    double x;
    double y;
```

```
};
```

Adesso `Punto` è un tipo a tutti gli effetti, come `int` o `double`. Possiamo creare variabili di quel tipo e accedere ai campi con il punto (`.`), usando la `dot notation`:

```
int main() {
    //opzione 1: dichiarazione e assegnazione separate
    Punto p;
    p.x = 3.0;
    p.y = 5.0;
    //opzione 2: inizializzazione compatta
    Punto q = {3.0, 4.0};

    cout << "x = " << p.x << ", y = " << p.y << "\n";
    // stampa: x = 3.0, y = 5.0
}
```

Analogia. Una `struct` è come una scheda anagrafica: il tipo `Persona` potrebbe avere i campi `nome`, `cognome`, `eta`. Ogni variabile di tipo `Persona` è una scheda contenente valori specifici.

2.3 Array in C++: sequenze di elementi dello stesso tipo

Per rappresentare più elementi dello stesso tipo in modo compatto, si usa un **array**. Un array è una sequenza di celle di memoria contigue, tutte dello stesso tipo, accessibili tramite un indice che parte da zero:

```
double valori[5] = {10.0, 20.0, 30.0, 40.0, 50.0};

// accesso:
cout << valori[0]; // stampa 10.0
cout << valori[4]; // stampa 50.0 (l'ultimo!)
```

Attenzione agli indici. Un array di 5 elementi ha indici da 0 a 4, *non* da 1 a 5. Accedere a `valori[5]` è un errore: C++ leggerebbe memoria che non appartiene all'array, per quanto sia contigua.

Per scorrere tutti gli elementi di un array si usa ad esempio un ciclo `for`:

```
int n = 5;
double valori[5] = {10.0, 20.0, 30.0, 40.0, 50.0};
for (int i = 0; i < n; i++) {
    cout << "valori[" << i << "] = " << valori[i] << "\n";
    // stampa ogni elemento preceduto dal suo indice
}
```

2.4 Costanti simboliche: `const` e `#define`

Negli array la dimensione deve essere nota a *tempo di compilazione*, ovvero deve essere un valore fisso. Per una questione di uniformità si preferisce premettere a ogni programma i valori costanti che saranno utilizzati uniformemente nel seguito.

```
const int DIM = 2; // numero di coordinate
const int MAX_PUNTI = 100; // quanti punti rappresento
double coordinate[DIM];     // array di 2 double
```

Cambiare `DIM` da 2 a 3 adatterà automaticamente tutto il codice che usa quella costante.

2.5 Array di `struct`

Nulla vieta di mettere un array come campo di una `struct`. Per il nostro classificatore, ogni punto non sarà nel piano bidimensionale, ma in uno spazio con `DIM` dimensioni (potrebbe essere 2, come la posizione; o 10, come i parametri che definisconoun frutto). Lo possiamo rappresentare così:

```
const int DIM = 2;
struct Punto {
    double features[DIM]; // le coordinate/misure
    int label; // la classe di appartenenza (es. 0=kiwi, 1=
        mela)
};
```

Terminologia. Nel gergo del machine learning, le misure numeriche che descrivono un oggetto si chiamano **features** (caratteristiche). L'etichetta che identifica la categoria si chiama **label** (etichetta). Un insieme di esempi con le relative etichette forma il **training set** (insieme di addestramento).

3. Passare dati a una funzione: puntatori e array

3.1 Le funzioni come scatole nere

Una funzione riceve degli input (*parametri*), fa qualcosa, e restituisce un output. Possiamo immaginarne una che calcola il quadrato di un numero:

```
double quadrato(double x) {
    return x * x;
}

int main() {
    double r = quadrato(5.0); // r = 25.0
}
```

Si faccia attenzione al fatto che x è un double e deve essere passato come tale.

3.2 Passaggio di array a una funzione

Quando passiamo un array a una funzione, non ne viene fatta una copia: viene passato l'indirizzo del primo elemento. La funzione lavora direttamente sull'array originale. Si parla in questo caso di passaggio del parametro per riferimento. Se invece passo il valore n, è un intero passato per valore.

La parola chiave `const` davanti al parametro garantisce che la funzione non modificherà l'array. È una "promessa" esplicita al compilatore (e a chi programma).

```
//dichiaro la funzione
double somma(const double arr[], int n) {
    double tot = 0.0;
    for (int i = 0; i < n; i++)
        tot += arr[i];

    return tot;
}

int main() {
    double valori[4] = {1.0, 2.0, 3.0, 4.0};
    //richiamo la funzione
    cout << somma(valori, 4); // stampa 10.0
}
```

3.3 Passaggio di `struct` a una funzione

Una `struct` può essere passata per valore (ne viene fatta una copia) oppure per riferimento costante (nessuna copia, nessuna modifica). Per strutture grandi, il secondo è preferibile:

```
void stampa_punto(const Punto& p) {
    cout << "(" << p.features[0] << ", " << p.features[1]
         << ") -> classe " << p.label << "\n";
}
```

La `&` indica il *passaggio per riferimento* (detto anche per indirizzo): la funzione lavora sull'oggetto originale, senza copiarlo.

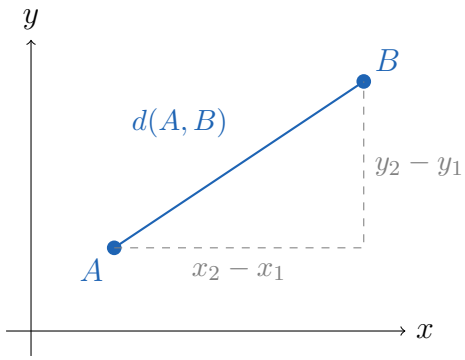
4. La distanza euclidea

4.1 Cos'è una distanza?

Prima di avvicinarci all'algoritmo, abbiamo bisogno di rispondere a una domanda fondamentale: *come si misura la "somiglianza" tra due punti?*

La risposta più naturale è la **distanza euclidea**, quella che intuitivamente associamo al righello. Nel piano (2 dimensioni), tra i punti $A = (x_1, y_1)$ e $B = (x_2, y_2)$ è il teorema di Pitagora:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



4.2 Generalizzazione a n dimensioni

Se ogni punto ha non due ma `DIM` coordinate, la formula si estende in modo naturale: si somma il quadrato della differenza per ciascuna coordinata e si estrae la radice:

$$d(A, B) = \sqrt{\sum_{i=0}^{\text{DIM}-1} (A_i - B_i)^2}$$

Non c'è niente di misterioso: è lo stesso Pitagora, applicato a più dimensioni. Avere più dimensioni permette di associare ad ognuna delle stesse una caratteristica diversa del frutto, come ad esempio peso, colore, superficie, ecc.

4.3 Implementazione in C++

Come implementare una distanza del genere in più dimensioni? Inizializzo la sum a zero, poi scorro le DIM coordinate cumulando la somma dei quadrati delle differente. Alla fine, calcolo la radice (per farlo includo cmath e uso la funzione `sqrt`).

```
#include <cmath>
const int DIM = 2;
double euclidean(const double a[], const double b[]) {
    double sum = 0.0;
    for (int i = 0; i < DIM; i++)
        sum += (a[i] - b[i]) * (a[i] - b[i]);
    return sqrt(sum);
}
```

Notate come il codice rispecchi quasi letteralmente la formula: si accumula la somma dei quadrati delle differenze, poi si restituisce la radice quadrata.

5. Ordinare dati: `sort` e `pair`

5.1 Il problema dell'ordinamento

Definiamo query, in questo contesto, il punto che vogliamo classificare. Per intenderci: prendo un frutto dalla cassetta, lo osservo, e voglio sapere se è una mela o un kiwi.

L'algoritmo KNN ha bisogno di trovare i K punti più vicini alla query. Il modo più semplice è quello di calcolare le distanze, ordinarle in senso crescente e prendere le prime K.

La libreria standard del C++ offre `sort`, che ordina un array sul posto:

```
#include <algorithm>

int main() {
    // array disordinato
    double arr[5] = {3.1, 1.4, 2.7, 0.9, 5.0};
    // ordina in-place
    sort(arr, arr + 5);
    // array ordinato: arr = {0.9, 1.4, 2.7, 3.1, 5.0}
}
```

I due parametri sono il puntatore al primo elemento e il puntatore all'elemento *dopo* l'ultimo (convenzione standard del C++).

5.2 Tenere insieme distanza ed etichetta: `pair`

Quando ordiniamo le distanze, dobbiamo tenere traccia di *quale punto* corrisponde a ciascuna distanza. Altrimenti dopo l'ordinamento non sapremo più a chi apparteneva ogni valore.

`pair` è una struttura predefinita della libreria standard che contiene esattamente due valori, chiamati `first` e `second`:

```
#include <utility> // per pair (spesso gia' incluso)

pair<double, int> p;
p.first  = 3.14;  // la distanza
p.second = 1;     // l'etichetta del punto

// oppure con inizializzazione compatta:
pair<double, int> q = {1.41, 0};

cout << q.first  << "\n"; // 1.41
cout << q.second << "\n"; // 0
```

Quando `sort` ordina un array di `pair`, usa di default il campo `first` come chiave: esattamente quello che vogliamo, dato che `first` contiene la distanza. Posso in

questo modo decidere di ordinare le distanze senza perdere l'associazione tra le etichette.

```
pair<double, int> dists[7];
// ... riempire dists ...
sort(dists, dists + 7);
// ora dists[0].first e' la distanza minima
// e dists[0].second e' l'etichetta del punto piu' vicino
```

6. Contare i voti: `map`

6.1 Il problema del conteggio

Una volta identificati i K vicini, dobbiamo contare quante volte compare ciascuna etichetta (classe) e scegliere quella più frequente. Questo si chiama *majority voting*.

Potremmo farlo con array doppi, ma la libreria standard offre uno strumento molto più elegante: `map`.

6.2 La `map`: un dizionario chiave-valore

Una `map` associa a ogni *chiave* un *valore*. È il classico dizionario: la chiave è la parola, il valore è la definizione.

Nel nostro caso, la chiave è l'etichetta (un `int`) e il valore è il conteggio (quante volte l'abbiamo vista):

```
#include <map>

map<int, int> votes; // chiave: etichetta, valore: conteggio

votes[0]++; // la classe 0 ottiene un voto
votes[1]++; // la classe 1 ottiene un voto
votes[0]++; // la classe 0 ottiene un altro voto

// votes[0] vale 2, votes[1] vale 1
```

Comodità della `map`. Se accediamo a una chiave che non esiste ancora, viene creata automaticamente con valore zero. Quindi `votes[label]++` funziona senza inizializzazione esplicita: il primo accesso crea la coppia `(label, 0)`, poi l'incremento la porta a 1.

6.3 Trovare il massimo in una `map`

Per trovare la chiave con il valore più alto usiamo `max_element`, passando un comparatore che confronta i valori (non le chiavi):

```
map<int, int> votes;
// ... riempire votes ...

int classe_vincitrice = max_element(
    votes.begin(), votes.end(),
    [](const pair<int,int>& a, const pair<int,int>& b) {
        return a.second < b.second; // confronta i conteggi
    }
)->first; // ->first prende la chiave (l'etichetta)
```

La sintassi `[](const pair<int,int>& a, ...)` viene detta funziona *lambda*, e si tratta di una funzione anonima definita e utilizzata sul posto.

Serve semplicemente a dire: "confronta due coppie guardando il secondo elemento (il conteggio)".

Se non ne facessimo uso, dovremmo scrivere molte righe di codice in più.

7. L'algoritmo KNN

7.1 Schema generale

L'algoritmo KNN si articola in tre passi:

1. **Calcola** la distanza euclidea tra il punto query e ciascun punto del training set. Memorizza ogni coppia (distanza, etichetta).
2. **Ordina** le coppie per distanza crescente. I primi K sono i vicini più prossimi.
3. **Vota**: conta le etichette dei K vicini e restituisci quella con più voti.

7.2 La funzione `knn_predict`

```
int knn_predict(const Point train[], int n,
                const double query[], int k) {

    // Passo 1: calcola le distanze
    pair<double, int> dists[MAX_POINTS];
    for (int i = 0; i < n; i++)
        dists[i] = {euclidean(train[i].features, query),
                    train[i].label};

    // Passo 2: ordina per distanza crescente
    sort(dists, dists + n);

    // Passo 3: voto di maggioranza tra i k vicini
    map<int, int> votes;
    for (int i = 0; i < k; i++)
        votes[dists[i].second]++;
```

```

    return max_element(votes.begin(), votes.end(),
        [](const pair<int,int>& a, const pair<int,int>& b) {
            return a.second < b.second;
        })->first;
}
```

7.3 Il programma completo

Ecco il codice finale, completo e compilabile. Si noti come ciascun componente emerga naturalmente da quanto discusso nelle sezioni precedenti:

```
#include <iostream>
#include <cmath>
#include <algorithm>
#include <map>

using namespace std;

const int MAX_POINTS = 100;
const int DIM = 2;

struct Point {
    double features[DIM];
    int label;
};

double euclidean(const double a[], const double b[]) {
    double sum = 0.0;
    for (int i = 0; i < DIM; i++)
        sum += (a[i] - b[i]) * (a[i] - b[i]);
    return sqrt(sum);
}

int knn_predict(const Point train[], int n,
                const double query[], int k) {
    pair<double, int> dists[MAX_POINTS];
    for (int i = 0; i < n; i++)
        dists[i] = {euclidean(train[i].features, query),
                    train[i].label};

    sort(dists, dists + n);

    map<int, int> votes;
    for (int i = 0; i < k; i++)
        votes[dists[i].second]++;

    return max_element(votes.begin(), votes.end(),
        [](const pair<int,int>& a, const pair<int,int>& b) {
            return a.second < b.second;
        })->first;
```

```
}

int main() {
    Point train[] = {
        {{1.0, 1.0}, 0}, {{1.5, 2.0}, 0}, {{3.0, 4.0}, 0},
        {{5.0, 7.0}, 1}, {{3.5, 5.0}, 1}, {{4.5, 5.0}, 1},
        {{3.5, 4.5}, 1}
    };
    int n = 7;

    double queries[][DIM] = {{1.2, 1.5}, {3.6, 5.0}, {4.0,
       4.0}};
    int nq = 3;

    int k = 3;
    for (int i = 0; i < nq; i++) {
        int pred = knn_predict(train, n, queries[i], k);
        cout << "Query (" << queries[i][0] << ", "
             << queries[i][1] << ") -> classe " << pred << "
                \n";
    }

    return 0;
}
```

8. Leggere l'output e capire cosa succede

Compilate ed eseguite il programma:

```
g++ -O2 -o knn knn.cpp && ./knn
```

L'output sarà:

```
Query (1.2, 1.5) -> classe 0
Query (3.6, 5.0) -> classe 1
Query (4.0, 4.0) -> classe 1
```

Vediamo cosa è successo per la prima query $(1.2, 1.5)$:

Punto nel training set	Etichetta	Distanza da (1.2, 1.5)
(1.0, 1.0)	0	0.54
(1.5, 2.0)	0	0.58
(3.0, 4.0)	0	3.25
(5.0, 7.0)	1	7.26
(3.5, 5.0)	1	4.08
(4.5, 5.0)	1	5.00
(3.5, 4.5)	1	3.74

Con $K = 3$, i tre vicini più prossimi sono: $(1.0, 1.0)$, $(1.5, 2.0)$, $(3.0, 4.0)$, tutti con etichetta 0. Risultato: **classe 0**. Corretto.

9. Scegliere K: un parametro con carattere

Il valore di K influenza il comportamento del classificatore in modo significativo:

- **K piccolo (es. K=1):** il classificatore è molto sensibile ai singoli punti. Un outlier (un punto anomalo) può distorcere la predizione. Alta varianza, bassa stabilità.
- **K grande (es. K=N):** si guarda a tutti i punti, il che equivale a restituire sempre la classe maggioritaria nel dataset. Troppo rozzo.
- **K dispari:** consigliato per evitare pareggi nel voto di maggioranza (con 2 classi).
- **K ottimale:** dipende dal problema. Si sceglie sperimentalmente, di solito con validazione incrociata.

Regola pratica. Come punto di partenza, molti libri suggeriscono $K \approx \sqrt{N}$, dove N è la dimensione del training set. Non è una legge, ma un buon punto di partenza prima di ottimizzare.

10. Limiti e sviluppi possibili

Questo classificatore è corretto ma essenziale. In un contesto reale si affronterebbero anche questi problemi:

1. **Normalizzazione delle features.** Se una coordinata vale centinaia e l'altra vale 0–1, la distanza sarà dominata dalla prima. Le features vanno portate sulla stessa scala.
2. **Complessità computazionale.** Con N punti e D dimensioni, ogni predizione costa $O(N \cdot D)$. Per dataset grandi si usano strutture come il *KD-tree*.
3. **Distanza pesata.** Invece di dare lo stesso peso a tutti i K vicini, si può pesare il loro voto inversamente alla distanza: chi è più vicino conta di più.
4. **Scelta della metrica.** La distanza euclidea è solo una delle possibili metriche. Esistono la distanza di Manhattan, quella di Minkowski, la similarità del coseno, e molte altre — ognuna adatta a diversi tipi di dati.

Conclusione

Abbiamo percorso un cammino che parte dai fondamentali del C++ — `struct`, array, funzioni, puntatori — e arriva a un classificatore funzionante.

Il classificatore KNN insegna qualcosa di importante: spesso le idee più eleganti dell'informatica nascono da intuizioni semplicissime. "Cerca ciò che ti assomiglia" è un principio antico quanto la cognizione umana. Il codice non fa altro che dargli una forma precisa e riproducibile.

Il codice che avete tra le mani è un classificatore reale. Non un giocattolo. Cambiate il training set, aumentate `DIM`, aggiungete classi: funzionerà. Questa è la soddisfazione di aver capito davvero qualcosa.